Complete Kubernetes Interview + Production Guide

Table of Contents

1. Kubernetes Basics

What is Kubernetes?

Kubernetes is a container orchestration platform used to:

- Deploy containers
- Scale applications
- Manage networking
- Handle self-healing
- Perform rolling updates
- Automate infrastructure operations

Why Kubernetes?

Problems without Kubernetes:

- Manual container management
- No autoscaling
- No self-healing
- Difficult deployments
- Networking complexity
- High operational overhead

Kubernetes solves:

- High availability
- Scaling
- Automated deployments
- Self-healing
- Load balancing
- Service discovery

Kubernetes Cluster

A Kubernetes cluster consists of:

1. Control Plane Nodes (Masters)
2. Worker Nodes

Example:

```
Control Plane:
master01
master02
master03

Worker Nodes:
```

```
worker01
worker02
worker03
```

2. Kubernetes Architecture

Control Plane Components

- API Server
- ETCD
- Scheduler
- Controller Manager

Worker Node Components

- kubelet
- kube-proxy
- Container Runtime
- CNI Plugin
- Pods

3. ETCD Deep Understanding

What is ETCD?

ETCD is a distributed key-value database.

It stores:

- Cluster state
- Deployment configs
- Secrets
- ConfigMaps
- Node information
- Service information
- Pod metadata

If ETCD is lost:

- Cluster state is lost
- Kubernetes cannot function properly

Where ETCD Resides?

In self-managed Kubernetes:

- ETCD usually runs on control plane nodes.

Example:

```
master01 -> etcd
master02 -> etcd
master03 -> etcd
```

ETCD Backup

Backup Command

```
ETCDCTL_API=3 etcdctl snapshot save backup.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

ETCDCTL_API=3 Meaning

ETCD API version.

Why TLS Certificates Needed?

ETCD communication is encrypted.

Certificates:

- ca.crt -> verifies server
- server.crt -> server identity
- server.key -> authentication

Even inside private GCP networks:

- TLS is still recommended
- Protects against internal attacks
- Required in most enterprise environments

Backup Script Example

```
#!/bin/bash

DATE=$(date +%F-%H-%M)
BACKUP_FILE="/backup/etcd-$DATE.db"
```

```
ETCDCTL_API=3 etcdctl snapshot save $BACKUP_FILE \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key

# Upload to GCS
gsutil cp $BACKUP_FILE gs://my-etcd-backups/
```

ETCD Quorum

Formula:

quorum = floor(n/2) + 1

Examples:

ETCD Nodes	Quorum	Failure Tolerance
1	1	0
3	2	1
5	3	2

Raft Consensus

ETCD uses Raft algorithm.

One node becomes leader. Leader replicates data to followers.

Write successful only after majority agrees.

4. API Server

What is API Server?

Central entry point for Kubernetes cluster.

All communication goes through API Server.

Example:

```
kubectl -> API Server
Scheduler -> API Server
Controller -> API Server
```

API Server HA

API Servers run behind Load Balancer.

Example:

```
kubectl
  ↓
Load Balancer VIP
  ↓
API Server 1
API Server 2
API Server 3
```

Default port:

```
6443
```

kubeconfig Example

File:

```
~/.kube/config
```

Example:

```yaml
clusters:
- name: prod-cluster
  cluster:
    server: https://prod-api.company.com:6443

contexts:
- name: prod-context
  context:
    cluster: prod-cluster
    user: admin

current-context: prod-context
```

Useful Commands

```
kubectl config current-context
kubectl config use-context prod-context
```

5. Scheduler

What Scheduler Does?

Scheduler decides:

- Which node pod should run on

Scheduler checks:

- Node CPU
- Node memory
- Taints/Tolerations
- Node affinity
- Resource requests
- Available ports

Scheduler Flow

```
Pod created
   ↓
Scheduler checks available nodes
   ↓
Best node selected
   ↓
Pod assigned to node
```

Resource Requests Example

```yaml
resources:
  requests:
    cpu: "1"
    memory: "2Gi"
```

Scheduler checks if node has:

- 1 CPU free
- 2Gi memory free

Taints and Tolerations

Taint

```
kubectl taint nodes node1 gpu=true:NoSchedule
```

Meaning:

- Only pods with matching toleration can run.

Toleration Example

```
tolerations:
- key: "gpu"
  operator: "Equal"
  value: "true"
  effect: "NoSchedule"
```

Scheduler HA

Multiple schedulers can run. Only one becomes active leader.

Leader election stored in ETCD.

6. Controller Manager

Controller Manager runs controllers:

- Deployment controller
- ReplicaSet controller
- Node controller
- Namespace controller

Responsibilities:

- Self-healing
- Node monitoring
- Replica maintenance

Node Failure Example

Suppose node dies.

Controller Manager detects:

```
Node NotReady
```

Then:

- Missing pods recreated on healthy nodes

7. kubelet

kubelet runs on every node.

Responsibilities:

- Starts containers
- Reports node status

- Reports pod metrics
- Talks to API server

kubelet Heartbeats

kubelet continuously sends node health.

If node stops reporting:

```
Node -> NotReady
```

8. kube-proxy

What kube-proxy Does?

kube-proxy handles:

- Service networking
- Service load balancing
- Port forwarding
- Service IP translation

kube-proxy Runs As

DaemonSet.

One pod per node.

kube-proxy Modes

iptables Mode

Uses Linux iptables rules.

Good for small-medium clusters.

IPVS Mode

Uses Linux IP Virtual Server.

Better performance. Better scalability. Preferred for large clusters.

Check IPVS

```
ipvsadm -L
```

Check iptables

```
iptables -L
```

kube-proxy Example

Service:

```
frontend-service
10.96.0.20
```

Backend Pods:

```
10.1.1.10
10.1.1.11
10.1.1.12
```

kube-proxy forwards:

```
10.96.0.20
  ↓
backend pods
```

9. CNI Plugin

CNI Full Form

Container Network Interface

What CNI Does?

- Pod IP assignment
- Pod routing
- Pod networking
- Overlay networking
- Network policies

Popular CNI Plugins

Plugin	Usage
Calico	Most popular
Cilium	Modern eBPF
Flannel	Simple
Weave	Older

Difference: CNI vs kube-proxy

Feature	CNI	kube-proxy
Pod IP allocation	YES	NO
Pod networking	YES	NO
Service routing	NO	YES
Load balancing	NO	YES

10. Kubernetes Networking

Three Major CIDRs

CIDR	Purpose
VPC CIDR	Node/VM IPs
Pod CIDR	Pod IPs
Service CIDR	Service IPs

Example

Resource	CIDR
VPC	10.0.0.0/16
Pod CIDR	10.1.0.0/16
Service CIDR	10.96.0.0/12

Pod Subnet Allocation

Example:

Node	Pod Subnet
worker01	10.1.1.0/24
worker02	10.1.2.0/24

Pod Networking Flow

```
frontend-pod
10.1.1.15
   ↓
backend-pod
10.1.2.18
```

Handled by:

- CNI plugin

11. Services

What is Service?

Service provides stable networking endpoint for pods.

Pods are temporary. Service IP remains stable.

Service Types

Type	Usage
ClusterIP	Internal only
NodePort	Exposes node port
LoadBalancer	Cloud LB
ExternalName	External DNS

Service Example

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend

  ports:
```

```
    - port: 80
      targetPort: 8080
```

Service Selector Example

Service:

```
selector:
  app: backend
```

Pod:

```
labels:
  app: backend
```

Service routes traffic to matching pods.

12. Ingress and Load Balancing

Ingress Resource

Ingress defines routing rules.

Example Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress

spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: backend-service
            port:
              number: 80
```

Ingress Controller

Ingress Controller is actual traffic processor.

Usually:

- NGINX
- HAProxy
- Traefik

Runs as pods.

Production Traffic Flow

```
User
 ↓
Cloud Load Balancer
 ↓
Ingress Controller Pod
 ↓
Service
 ↓ kube-proxy
Backend Pod
```

13. HPA

HPA Full Form

Horizontal Pod Autoscaler

HPA Purpose

Automatically scales pod replicas.

HPA Formula

desiredReplicas = currentReplicas × currentCPU / targetCPU

Example

Current replicas = 3 Current CPU = 85% Target CPU = 70%

Calculation:

3 × 85 / 70 = 4

Pods scaled:

```
3 -> 4
```

HPA YAML Example

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: frontend-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: frontend-app

  minReplicas: 3
  maxReplicas: 10

  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Metrics Server

Metrics Server collects:

- CPU usage
- Memory usage

from kubelets.

Runs inside cluster.

Usually in:

```
kube-system namespace
```

Metrics Flow

```
Container
 ↓
Linux cgroups
 ↓
kubelet
 ↓
Metrics Server
 ↓
HPA
```

Check Metrics

```
kubectl top nodes
kubectl top pods
```

14. Cluster Autoscaler

Purpose

Automatically scales worker nodes.

Flow

```
HPA creates more pods
   ↓
No node capacity
   ↓
Pods Pending
   ↓
Cluster Autoscaler adds worker nodes
```

15. DaemonSet

What is DaemonSet?

Ensures one pod runs on every node.

Common Use Cases of DaemonSet

Tool	Purpose
node-exporter	Node metrics
FluentD	Log collection
Datadog Agent	Monitoring
Falco	Security monitoring
Filebeat	Log shipping

DaemonSet Example

```yaml
apiVersion: apps/v1
kind: DaemonSet

metadata:
  name: node-exporter

spec:
  selector:
    matchLabels:
      app: node-exporter

  template:
    metadata:
      labels:
        app: node-exporter

    spec:
      containers:
      - name: node-exporter
        image: prom/node-exporter:latest

        ports:
        - containerPort: 9100
```

DaemonSet YAML Explanation

Field	Meaning
kind: DaemonSet	One pod per node
selector	Identifies matching pods
template	Pod template
containerPort: 9100	Metrics exposed on port 9100

How DaemonSet Works Internally

Suppose cluster has:

```
worker01
worker02
worker03
```

DaemonSet automatically creates:

```
node-exporter pod on worker01
node-exporter pod on worker02
node-exporter pod on worker03
```

If:

- new node added

Then:

- Kubernetes automatically deploys DaemonSet pod there.

16. PV vs PVC Deep Understanding

PV Full Form

```
Persistent Volume
```

Actual storage resource.

Examples:

- GCE Persistent Disk
- AWS EBS
- Azure Disk
- NFS
- Ceph

PVC Full Form

```
Persistent Volume Claim
```

PVC is:

- request for storage

Application does NOT directly use PV.

Application requests PVC.

PVC binds to PV.

Real Production Example

Suppose:

- MySQL database needs 100GB storage

Application YAML:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: mysql-pvc

spec:
  accessModes:
    - ReadWriteOnce
```

```
  resources:
    requests:
      storage: 100Gi
```

Kubernetes Flow

```
Application
   ↓
PVC Request
   ↓
PV Bound
   ↓
Actual Disk Attached
```

Why PVC Needed?

Without PVC:

- applications tightly coupled to storage

PVC provides:

- abstraction
- portability
- dynamic provisioning

PV Example

```yaml
apiVersion: v1
kind: PersistentVolume

metadata:
  name: mysql-pv

spec:
  capacity:
    storage: 100Gi

  accessModes:
    - ReadWriteOnce

  gcePersistentDisk:
    pdName: mysql-disk
    fsType: ext4
```

Difference Between PV and PVC

PV	PVC
Actual storage	Storage request
Created by admin	Created by app/team
Physical resource	Logical request

Access Modes

Mode	Meaning
ReadWriteOnce	One node read/write
ReadOnlyMany	Multiple read only
ReadWriteMany	Multiple read/write

StorageClass

StorageClass enables:

- dynamic disk provisioning

Example:

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass

metadata:
  name: fast-ssd

provisioner: kubernetes.io/gce-pd

parameters:
  type: pd-ssd
```

Dynamic Provisioning Flow

```
PVC created
   ↓
StorageClass triggered
   ↓
Cloud disk auto-created
   ↓
PV auto-created
   ↓
PVC bound
```

17. Monitoring and Logging Deep Understanding

Why Monitoring Needed?

Monitor:

- CPU
- memory
- disk
- pod crashes
- API latency
- node failures
- application metrics

Monitoring Stack

Most common:

```
Prometheus + Grafana
```

Components

Component	Purpose
Prometheus	Metrics collection
Grafana	Dashboard
Alertmanager	Alerts
node-exporter	Node metrics
kube-state-metrics	Kubernetes metrics

Metrics Collection Flow

```
Node/Pod
   ↓
Exporter
   ↓
Prometheus
   ↓
Grafana
```

node-exporter

Runs as:

- DaemonSet

Collects:

- CPU
- RAM
- filesystem
- network

from Linux node.

kube-state-metrics

Collects:

- deployments
- replicas
- pod states
- job states

from Kubernetes API.

Logging Stack

Most common:

```
EFK / ELK
```

Components

Component	Purpose
Elasticsearch	Storage/search
FluentD/Filebeat	Log collection
Kibana	Visualization

Logging Flow

```
Pods
 ↓
Container Logs
```

```
 ↓
FluentD/Filebeat
 ↓
Elasticsearch
 ↓
Kibana
```

Production Monitoring Best Practices

- Monitor ETCD latency
- Monitor API server
- Monitor pod restarts
- Monitor disk pressure
- Monitor memory pressure
- Alert on node failures
- Alert on CrashLoopBackOff
- Monitor ingress latency

Important Monitoring Commands

```
kubectl top nodes
kubectl top pods
kubectl get events
kubectl describe node NODE_NAME
```

18. HA Kubernetes Architecture Deep Understanding

Production HA Architecture

```
              Users
                ↓
       Cloud Load Balancer
                ↓
       API Server VIP
       /     |      \
 master01 master02 master03
   ↓         ↓         ↓
  ETCD      ETCD      ETCD
```

Why HA Needed?

Avoid:

- single point of failure

If:

- one master dies

Cluster still operational.

Components on Each Master

Component	Runs on Master?
API Server	YES
ETCD	YES
Scheduler	YES
Controller Manager	YES

API Server HA

All API servers:

- active simultaneously

Load balancer distributes requests.

Scheduler HA

Multiple schedulers exist.

BUT:

- only one active leader

Others standby.

Why Only One Scheduler Leader?

To avoid:

- duplicate pod scheduling

Leader election stored in ETCD.

Check Scheduler Leader

```
kubectl get endpoints kube-scheduler -n kube-system -o yaml
```

Controller Manager HA

Same concept:

- one active leader

ETCD HA

Example:

```
3 ETCD nodes
```

Quorum:

```
2 required
```

Important ETCD Rule

Always use:

- odd number nodes

Example:

- 3
- 5
- 7

NEVER:

- 2
- 4
- 6

Why Odd Number?

Example:

3 nodes:

- quorum = 2

If 1 fails:

- still 2 alive

Cluster works.

If 2 Nodes Only

quorum = 2

If 1 fails:

- quorum lost

Cluster broken.

ETCD Quorum Example

Suppose:

```
master01 alive
master02 alive
master03 dead
```

Still:

- quorum = 2

Cluster operational.

What Happens When Deployment Created?

```
kubectl apply -f app.yaml
```

Flow:

```
kubectl
 ↓
API Server
 ↓
ETCD write
 ↓
Majority ETCD agrees
 ↓
Scheduler notified
 ↓
Pod assigned
 ↓
kubelet starts container
```

Important Clarification

Even if request reaches:

- only one API server

That API server:

- communicates with ETCD cluster

ETCD internally replicates data.

Raft Consensus Algorithm

Raft ensures:

- all ETCD nodes synchronized

Leader handles writes.

Followers replicate data.

Write successful only after:

- majority acknowledgement

19. Rolling Update vs Recreate

Rolling Update

Pods replaced gradually.

No downtime.

Example

Old pods:

```
v1-pod1
v1-pod2
v1-pod3
```

Rolling update:

```
create v2-pod1
delete v1-pod1

create v2-pod2
delete v1-pod2
```

Application remains available.

Recreate Strategy

Deletes all old pods first.

Then creates new pods.

Possible downtime.

Deployment Strategy YAML

```yaml
strategy:
  type: RollingUpdate
```

OR

```yaml
strategy:
  type: Recreate
```

Rolling Update Commands

```
kubectl rollout status deployment frontend
kubectl rollout history deployment frontend
kubectl rollout undo deployment frontend
```

20. Kubernetes Security Deep Understanding

Major Security Areas

Area	Purpose
RBAC	Access control
NetworkPolicy	Network restriction
Secrets	Credential management
TLS	Encryption
Pod Security	Restrict containers
Image Scanning	Vulnerability scanning

RBAC

Role-Based Access Control.

Controls:

- who can do what

Example

Developer:

- can view pods
- cannot delete nodes

Role Example

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role

metadata:
  name: pod-reader

rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

RoleBinding Example

```yaml
kind: RoleBinding
```

Assigns role to:

- user
- service account

NetworkPolicy

Controls pod-to-pod traffic.

Example:

- frontend can talk backend
- backend cannot talk frontend

NetworkPolicy Example

```yaml
kind: NetworkPolicy
```

Secrets

Store:

- passwords
- API keys
- certificates

Secret Example

```
kubectl create secret generic db-secret \
--from-literal=password=mypassword
```

Pod Security

Restrict:

- privileged containers
- root users
- host networking

Image Scanning

Scan images for:

- CVEs
- malware
- vulnerabilities

Tools:

- Trivy
- Clair

Service Accounts

Pods can use:

- Kubernetes Service Accounts

Cloud:

- GCP service accounts
- IAM roles

21. Helm Deep Understanding

What is Helm?

Helm is:

- package manager for Kubernetes

Like:

- apt for Ubuntu
- yum for RHEL

Problem Without Helm

Many YAML files:

```
deployment.yaml
service.yaml
ingress.yaml
configmap.yaml
secret.yaml
hpa.yaml
```

Difficult management.

Helm Solves

- templating
- versioning
- easier deployments
- reusable charts

Helm Chart Structure

```
mychart/
  templates/
  values.yaml
  Chart.yaml
```

values.yaml

Contains configurable values.

Example:

```
replicaCount: 3

image:
  repository: nginx
  tag: latest
```

Template Example

```
replicas: {{ .Values.replicaCount }}
```

Helm Commands

```
helm install myapp ./chart

helm upgrade myapp ./chart

helm rollback myapp 1

helm list
```

Helm Benefits

- reusable deployments
- environment consistency
- simpler upgrades

22. Operators Deep Understanding

What is Operator?

Operator:

- extends Kubernetes capabilities

Uses:

- custom controllers
- CRDs

CRD Full Form

Custom Resource Definition

Operator Use Cases

- databases
- Kafka
- Redis
- Elasticsearch

Example

MySQL Operator can:

- backup database
- recover database
- failover automatically
- scale replicas
- upgrade cluster

Why Operators Powerful?

Traditional deployment:

- only container lifecycle

Operator:

- understands application logic

Example

If MySQL primary fails:

- Operator promotes replica automatically

Popular Operators

Operator	Purpose
Prometheus Operator	Monitoring
MySQL Operator	MySQL HA
Elastic Operator	Elasticsearch
Kafka Operator	Kafka management

23. Production Troubleshooting Deep Understanding

Scenario 1: Pod Pending

Possible Reasons

Reason	Meaning
insufficient CPU	node lacks CPU
insufficient memory	node lacks RAM
taints	pod not tolerated
PVC issue	storage unavailable
node selector mismatch	wrong labels

Debug Commands

```
kubectl describe pod POD_NAME
kubectl get events
```

Example

```
0/5 nodes available: insufficient cpu
```

Meaning:

- no worker node has enough free CPU.

Scenario 2: CrashLoopBackOff

Meaning:

- container continuously crashing.

Common Reasons

Reason	Example
bad env variable	wrong DB hostname
DB unreachable	connection refused

Reason	Example
app crash	code bug
bad image	startup failure
missing secret	credentials absent

Debugging

```
kubectl logs POD_NAME
kubectl describe pod POD_NAME
```

Wrong Environment Variable Example

Application expects:

```
MYSQL_HOST=mysql-service
```

But YAML has:

```
MYSQL_HOST=mysql-port
```

Application cannot connect. Container crashes.

Scenario 3: Service Not Reachable

Check:

Check	Command
Service exists	kubectl get svc
Endpoints exist	kubectl get endpoints
Labels match	kubectl describe svc
Pod running	kubectl get pods

Selector Mismatch Example

Service selector:

```
selector:
  app: backend
```

Pod label:

```
labels:
  app: api
```

Service finds:

- ZERO pods

Traffic fails.

Scenario 4: Node NotReady

Causes

- kubelet stopped
- disk full
- network issue
- node crashed

Debug

```
kubectl describe node NODE_NAME
systemctl status kubelet
```

Scenario 5: DNS Failure

Symptoms:

- services unreachable by name

Check CoreDNS

```
kubectl get pods -n kube-system
```

Restart CoreDNS

```
kubectl rollout restart deployment coredns -n kube-system
```

24. Important kubectl Commands Deep Understanding

Cluster Commands

```
kubectl cluster-info
kubectl get nodes
kubectl describe node NODE_NAME
kubectl top nodes
```

Pod Commands

```
kubectl get pods
kubectl get pods -A
kubectl get pods -n kube-system
kubectl describe pod POD_NAME
kubectl logs POD_NAME
kubectl exec -it POD_NAME -- /bin/bash
kubectl top pods
```

Deployment Commands

```
kubectl get deployment
kubectl describe deployment DEPLOYMENT_NAME

kubectl scale deployment frontend --replicas=5

kubectl rollout status deployment frontend

kubectl rollout undo deployment frontend
```

Kubernetes `kind` Complete Guide with Sample YAML Files

1. Pod

Purpose

Smallest executable unit in Kubernetes.

Runs:

- one or more containers

Sample YAML

```yaml
apiVersion: v1

kind: Pod

metadata:
  name: nginx-pod
  labels:
    app: nginx

spec:
  containers:
  - name: nginx-container
    image: nginx:latest

    ports:
    - containerPort: 80

    resources:
      requests:
        cpu: "200m"
        memory: "256Mi"

      limits:
        cpu: "500m"
        memory: "512Mi"
```

2. Deployment

Purpose

Manages:

- replicas
- rolling updates
- self-healing

Sample YAML

```yaml
apiVersion: apps/v1

kind: Deployment

metadata:
  name: frontend-deployment

spec:
  replicas: 3

  selector:
    matchLabels:
      app: frontend

  template:
```

```
    metadata:
      labels:
        app: frontend

    spec:
      containers:
      - name: frontend-container
        image: nginx:latest

        ports:
        - containerPort: 80

        env:
        - name: ENV
          value: production

        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"

          limits:
            cpu: "1"
            memory: "1Gi"
```

3. StatefulSet

Purpose

Used for:

- databases
- Kafka
- Elasticsearch

Sample YAML

```
apiVersion: apps/v1

kind: StatefulSet

metadata:
  name: mysql-statefulset

spec:
  serviceName: mysql-service

  replicas: 3

  selector:
    matchLabels:
      app: mysql

  template:
    metadata:
      labels:
        app: mysql
```

```
    spec:
      containers:
      - name: mysql

        image: mysql:8

        env:
        - name: MYSQL_ROOT_PASSWORD
          value: password123

        ports:
        - containerPort: 3306

        volumeMounts:
        - name: mysql-storage
          mountPath: /var/lib/mysql

  volumeClaimTemplates:
  - metadata:
      name: mysql-storage

    spec:
      accessModes:
      - ReadWriteOnce

      resources:
        requests:
          storage: 10Gi
```

4. DaemonSet

Purpose

Runs:

- one pod per node

Used for:

- monitoring
- logging

Sample YAML

```
apiVersion: apps/v1

kind: DaemonSet

metadata:
  name: monitoring-agent

spec:
  selector:
    matchLabels:
      app: monitoring-agent
```

```
  template:
    metadata:
      labels:
        app: monitoring-agent

    spec:
      containers:
      - name: datadog-agent

        image: datadog/agent:latest

        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"

          limits:
            cpu: "500m"
            memory: "512Mi"
```

5. Job

Purpose

One-time execution.

Examples:

- backup
- migration

Sample YAML

```
apiVersion: batch/v1

kind: Job

metadata:
  name: backup-job

spec:
  template:
    spec:
      containers:
      - name: backup

        image: busybox

        command:
        - /bin/sh
        - -c
        - echo "Running backup"

      restartPolicy: Never
```

6. CronJob

Purpose

Scheduled jobs.

Sample YAML

```yaml
apiVersion: batch/v1

kind: CronJob

metadata:
  name: nightly-backup

spec:
  schedule: "0 2 * * *"

  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup

            image: busybox

            command:
            - /bin/sh
            - -c
            - echo "Nightly backup"

          restartPolicy: OnFailure
```

7. Service

Purpose

Exposes pods.

Provides:

- stable networking

Sample YAML

```yaml
apiVersion: v1

kind: Service

metadata:
  name: frontend-service
```

```
spec:
  type: ClusterIP

  selector:
    app: frontend

  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

8. LoadBalancer Service

Purpose

External access.

Sample YAML

```
apiVersion: v1

kind: Service

metadata:
  name: frontend-loadbalancer

spec:
  type: LoadBalancer

  selector:
    app: frontend

  ports:
  - port: 80
    targetPort: 8080
```

9. Ingress

Purpose

HTTP/HTTPS routing.

Sample YAML

```
apiVersion: networking.k8s.io/v1

kind: Ingress
```

```
metadata:
  name: app-ingress

spec:
  rules:
  - host: app.company.com

    http:
      paths:
      - path: /api
        pathType: Prefix

        backend:
          service:
            name: backend-service

            port:
              number: 80
```

10. PersistentVolume (PV)

Purpose

Actual storage resource.

Sample YAML

```
apiVersion: v1

kind: PersistentVolume

metadata:
  name: mysql-pv

spec:
  capacity:
    storage: 20Gi

  accessModes:
  - ReadWriteOnce

  persistentVolumeReclaimPolicy: Retain

  hostPath:
    path: /data/mysql
```

11. PersistentVolumeClaim (PVC)

Purpose

Storage request.

Sample YAML

```yaml
apiVersion: v1

kind: PersistentVolumeClaim

metadata:
  name: mysql-pvc

spec:
  accessModes:
  - ReadWriteOnce

  resources:
    requests:
      storage: 10Gi
```

12. StorageClass

Purpose

Defines storage provisioning.

Sample YAML

```yaml
apiVersion: storage.k8s.io/v1

kind: StorageClass

metadata:
  name: standard-storage

provisioner: kubernetes.io/gce-pd

parameters:
  type: pd-balanced
```

13. ConfigMap

Purpose

Stores configuration data.

Sample YAML

```yaml
apiVersion: v1

kind: ConfigMap
```

```
metadata:
  name: app-config

data:
  APP_ENV: production
  APP_DEBUG: "false"
```

14. Secret

Purpose

Stores:

- passwords
- API keys
- certificates

Sample YAML

```
apiVersion: v1

kind: Secret

metadata:
  name: db-secret

type: Opaque

data:
  username: YWRtaW4=
  password: cGFzc3dvcmQ=
```

Note:

- values are base64 encoded

15. HorizontalPodAutoscaler (HPA)

Purpose

Automatically scales pods.

Sample YAML

```
apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler
```

```yaml
metadata:
  name: frontend-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: frontend-deployment

  minReplicas: 3
  maxReplicas: 10

  metrics:
  - type: Resource

    resource:
      name: cpu

      target:
        type: Utilization
        averageUtilization: 70
```

HPA Formula

desired replicas:

:contentReference[oaicite:0]{index=0}

16. Namespace

Purpose

Logical isolation.

Sample YAML

```yaml
apiVersion: v1

kind: Namespace

metadata:
  name: production
```

17. NetworkPolicy

Purpose

Controls pod communication.

Sample YAML

```yaml
apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:
  name: allow-frontend-to-backend

spec:
  podSelector:
    matchLabels:
      app: backend

  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
```

18. Role

Purpose

Namespace-level permissions.

Sample YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1

kind: Role

metadata:
  namespace: default
  name: pod-reader

rules:
- apiGroups: [""]
  resources: ["pods"]

  verbs:
  - get
  - list
  - watch
```

19. RoleBinding

Purpose

Assign role to user/service account.

Sample YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding

metadata:
  name: read-pods

subjects:
- kind: User
  name: dev-user

roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

20. ClusterRole

Purpose

Cluster-wide permissions.

Sample YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRole

metadata:
  name: cluster-admin-role

rules:
- apiGroups: [""]

  resources:
  - pods
  - nodes
  - services

  verbs:
  - get
  - list
  - watch
```

21. ClusterRoleBinding

Purpose

Assign ClusterRole.

Sample YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRoleBinding

metadata:
  name: cluster-admin-binding

subjects:
- kind: User
  name: admin-user

roleRef:
  kind: ClusterRole
  name: cluster-admin-role
  apiGroup: rbac.authorization.k8s.io
```

22. ServiceAccount

Purpose

Identity for pods.

Sample YAML

```yaml
apiVersion: v1

kind: ServiceAccount

metadata:
  name: app-service-account
```

23. Full Production Flow

```
User
 ↓
Load Balancer
 ↓
Ingress
 ↓
Service
 ↓
Deployment
 ↓
```

```
ReplicaSet
 ↓
Pods
```

Important Interview Understanding

Resource	Creates Pods?
Pod	YES
Deployment	YES
StatefulSet	YES
DaemonSet	YES
Job	YES
CronJob	YES
Service	NO
Ingress	NO
ConfigMap	NO
Secret	NO
HPA	NO
PVC	NO

Important Kubernetes Categories

Workload Resources

```
Pod
Deployment
StatefulSet
DaemonSet
Job
CronJob
```

Networking Resources

```
Service
Ingress
NetworkPolicy
```

Storage Resources

```
PersistentVolume
PersistentVolumeClaim
StorageClass
```

Security Resources

```
Role
RoleBinding
ClusterRole
ClusterRoleBinding
ServiceAccount
```

Config Resources

```
ConfigMap
Secret
```

Scaling Resources

```
HorizontalPodAutoscaler
```

Final Important Concept

When you see:

```
kind: Something
```

It means:

```
"Kubernetes, create/manage this type of object."
```

It does NOT always mean:

- pod
- VM
- container

Sometimes it is:

- networking object
- storage object
- security object

- scaling object
- configuration object

This distinction is extremely important for Kubernetes interviews.

End of README